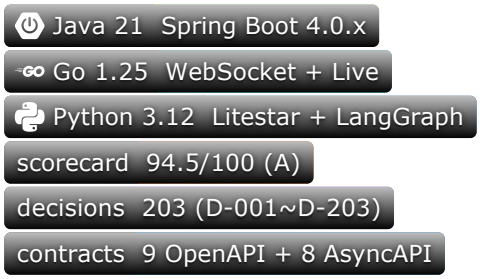
education-platform · 多语言云原生在线教育平台

> **一个三语言（Java 21 / Go 1.25 / Python 3.12）微服务教育 SaaS 平台**：14 个 Java 服务 + 2 个 Go 实时服务 + 1 个 Python AI 平台（9 模块）+ 7 个前端应用 + 7 个公共库，覆盖 **K12 在线教育、职业教育、棋类教育、康养养老** 四大业务板块，按 **4 单元（Cell）异地多活** 架构设计，SDD 契约先行（9 OpenAPI + 8 AsyncAPI），10 条工程铁律治理，**203 个架构决策（D-001 ~ D-203）全程留痕，架构评分卡 94.5 / A**。



目录

- 1. 项目概述
- 2. 系统架构
- 3. 核心设计决策
- 4. 关键实现剖析
- 5. 目录结构
- 6. 快速开始
- 7. 测试体系
- 8. 部署
- 9. 监控运维
- 10. 文档索引
- 11. 项目治理：铁律与决策

---

1. 项目概述

1.1 项目定位

本项目是一个**面向多租户 SaaS 的教育平台参考实现**，完整模拟大厂级全栈工程流程：

```
需求（SRS） → 架构（SAD/ADR） → 契约（SDD：OpenAPI/AsyncAPI） → 实现（三语言微服务）
        → 测试（1300+ Java 用例 / 变异测试 / k6 压测 / E2E） → 部署（K3s + ArgoCD GitOps + 4 单元多活）
        → 度量（架构评分卡 94.5/A） → 治理（10 条铁律 + 203 项决策留痕）
```

四大业务板块 × 八大服务域：

业务板块	服务域	承载服务	亮点
K12 / 职业教育	核心教学	`core-business`	课程/用户/报名、领域事件驱动、虚拟线程
K12 / 职业教育	智能批改	`grading-service` + `grading-api`	Judge0 沙箱执行、AI 查重、**Disruptor 三级批改流水线**
全板块	交易履约	`order-platform` + `payment-platform`	**Seata Saga 补偿事务**、幂等键、中台化独立抽取
全板块	知识库	`knowledge-bff` / `knowledge-service` / `retrieval-service` / `agent-service` / `mcp-server`	文档解析→向量化→检索管道、Milvus 语义检索、**MCP + A2A Agent 架构**
全板块	AI 平台	`ai-platform`（Python，9 模块）	**四层成本栈**（语义缓存→RouteLLM→级联→降级）、RAG、Reflexion
全板块	SaaS 控制平面	`tenant-service`	多租户隔离 + **Metering / Billing / ApiKey** 计量计费
棋类教育	棋牌对弈	`boardgame-service`	**象棋规则引擎** + Elo 天梯 + 走子流水线
康养养老	康养监护	`senior-care`	IoT 健康遥测、Disruptor 窗口聚合异常检测
全板块	实时互动	`ws-gateway` / `live-go`（Go）	**纯标准库 RFC6455 WebSocket 网关**、跨 Pod 广播、直播推拉流
全板块	数据平台	`data-platform`	血缘/剖析/漂移检测/NL Query、CDC 摄入流水线

1.2 关键数字

```
架构评分          94.5 / 100（A 级）
架构决策          203 项（D-001 ~ D-203，decisions.jsonl 全程留痕）
后端服务          17 个：Java 14 + Go 2 + Python 1（AI 平台内含 9 模块）
公共库            7 个：Java 5（ep-web-common / ep-disruptor / ep-cache / ep-data / ep-memory-java）
                        + Go 1（ep-common-go）+ Python 1（ep-common-python）
前端应用          7 个（Web 3 + H5 2 + 小程序 2），pnpm monorepo + @ep/ scope
SDD 契约          17 份（OpenAPI 9 + AsyncAPI 8），Spectral lint 门禁
Java 测试         ≈ 1300 用例 + 21 个 IT 类（Testcontainers：TiDB/CRDB/MySQL）
Python 测试       189 用例（pytest）
Go 测试           全覆盖（-race 跑法）
k6 压测           12 套脚本（1K → 百万级在线模拟、秒杀尖峰、跨语言对比）
CI Workflows      14 条（测试矩阵/覆盖率/变异测试/AOT 门禁/铁律扫描/三重安全扫描）
多活单元          4 个（cell-bj / cell-hz / cell-sz / cell-sg）
```

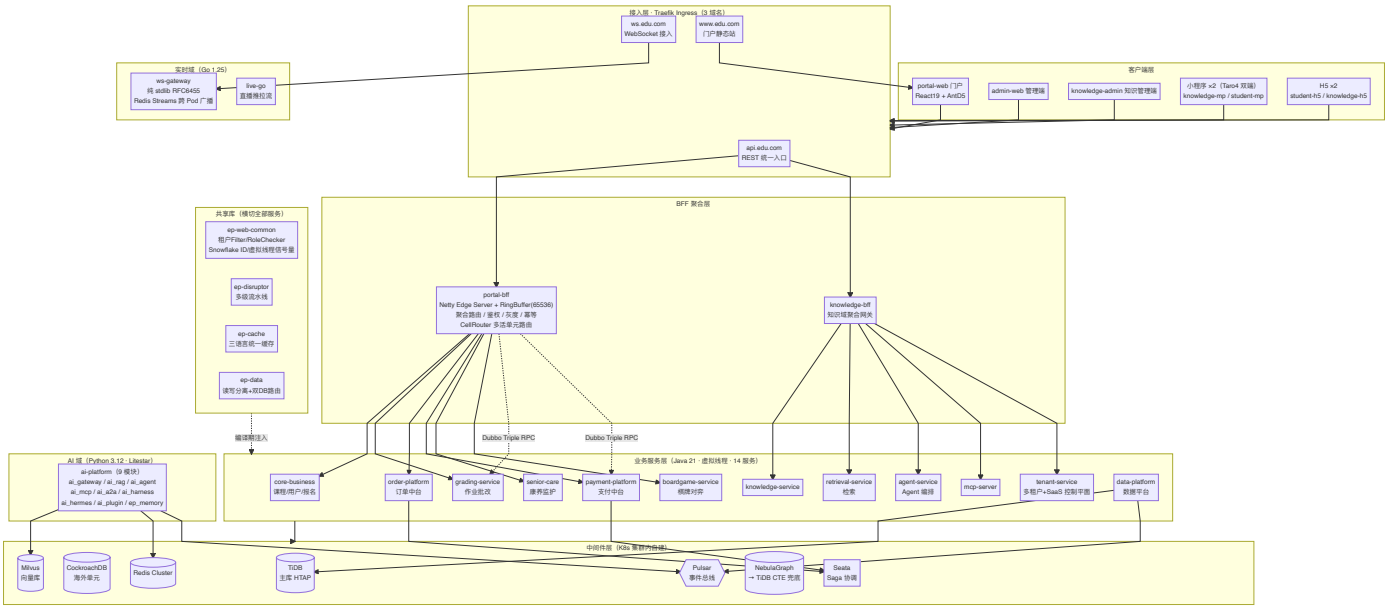
1.3 技术栈总表

层	选型	说明
Java 运行时	JDK 21，虚拟线程全局开启	IO 密集服务最优并发模型；配套 `DbConnectionSemaphore` 信号量防止虚拟线程打满连接池
Java 框架	Spring Boot 4.0.x（根 BOM 统一版本）	全 14 服务版本对齐；GraalVM AOT 元数据门禁
Java RPC	Apache Dubbo 3.3（Triple 协议）	IDL 友好、跨语言互通
分布式事务	Seata 2.0（**Saga 模式**）	订单-支付-库存长事务补偿（§4.1）
高并发内核	**LMAX Disruptor 4.0**	无锁 RingBuffer 多级流水线，公共库 `ep-disruptor`（§3.1）
数据访问	**ep-data**（可插拔 + 读写分离）	TiDB / CockroachDB / MySQL / H2 一键切换（§3.2）
缓存	**ep-cache**：L1 Caffeine + L2 Redis + **Pulsar 三语言失效广播**	Java/Go/Python 缓存一致性协议统一（§3.5）
韧性	Resilience4j（限流/断路）	多服务接入
Go 运行时	Go 1.25	goroutine per connection + channel 泵模型
Go WebSocket	**纯标准库 RFC6455 实现**	ws-gateway 零三方 WS 依赖；生产 fail-closed 鉴权
Python 运行时	Python 3.12 + asyncio	Litestar（非 FastAPI）+ Granian 服务器
LLM 编排	LangGraph + LangChain-Core +	Agent 状态机、四级记忆体系

	mem0ai	
消息	Apache Pulsar 3.x	领域事件总线 + 缓存失效 + WS 跨 Pod 广播
关系库	**TiDB（主）+ CockroachDB（海外单元）**	双库架构经 PoC 评审（D-194 / D-203）
向量库	Milvus（Java SDK 2.5 / pymilvus）	语义检索 + RAG
图库	NebulaGraph → **TiDB CTE 兜底**	知识图谱/血缘图，降级路径内置
前端	React 19 + AntD 5 + TanStack Query + Vite	pnpm monorepo + Turborepo
跨端	Taro 4（微信小程序 + H5 双端产出）	ui-kit 平台适配层解耦
可观测	OpenTelemetry + Micrometer	traceparent 全链路透传
构建	Maven / **JIB 免 Dockerfile 镜像** / pnpm + turbo	镜像构建确定性
GitOps	**ArgoCD app-of-apps** + Kustomize + Karmada	声明式多集群同步

2. 系统架构

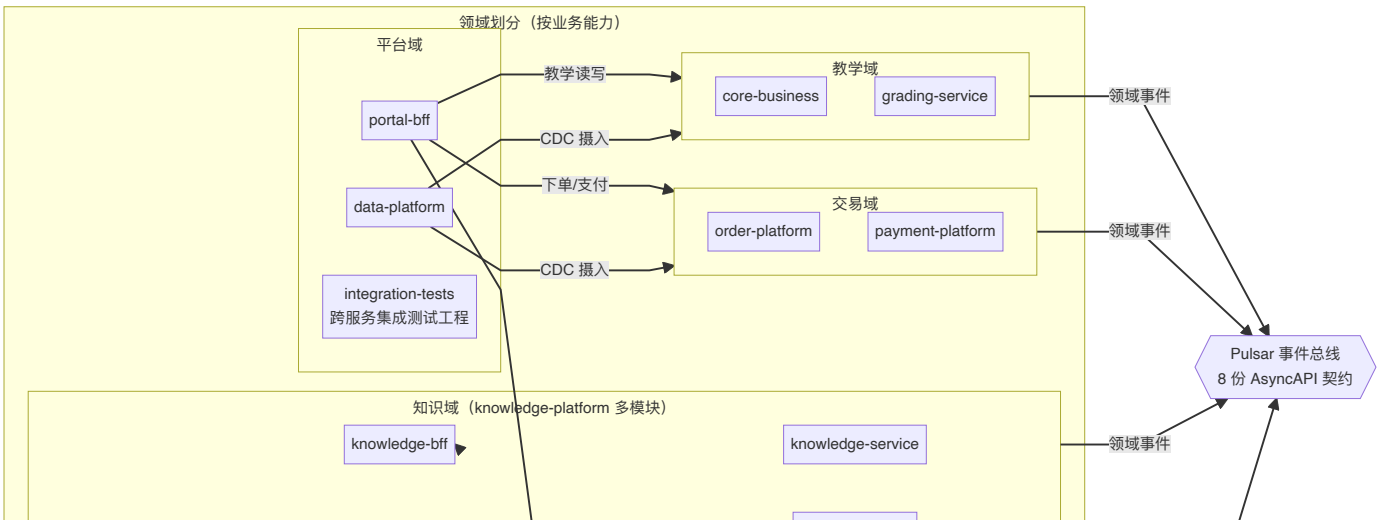
2.1 总体架构（客户端 → 网关 → BFF → 服务 → 中间件）

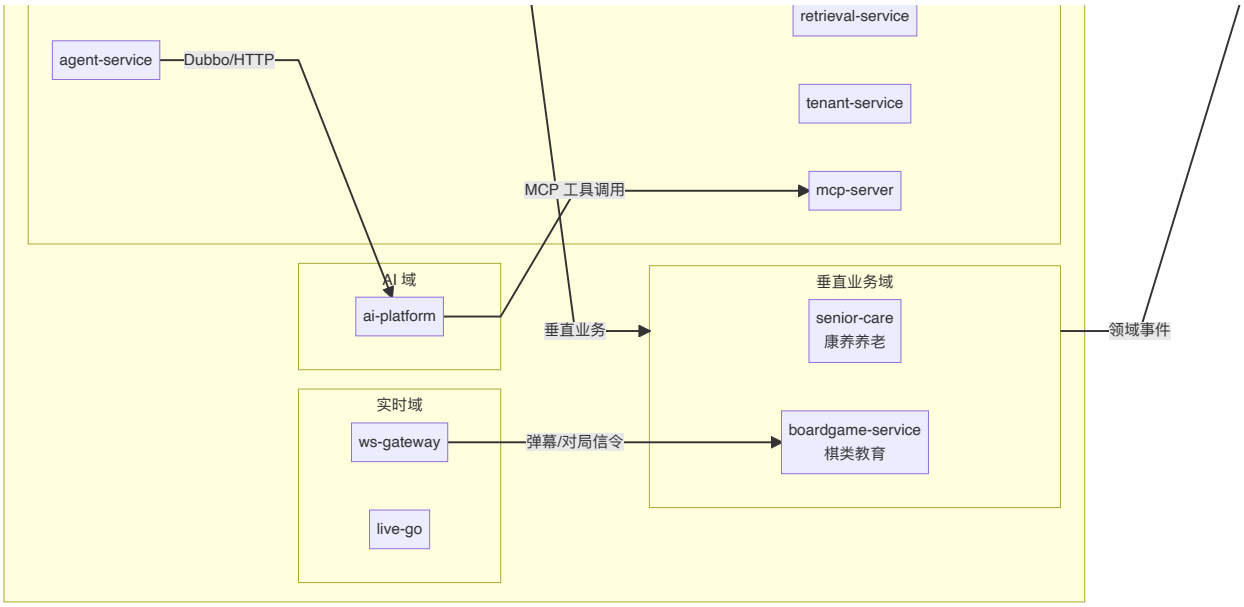

接入层 · Traefik Ingress（3 组名）
ws.edu.com
WebSocket 接入
www.edu.com
门户静态站
实时层（Go 1.25）
ws-gateway
纯 stdlib RFC6455
Redis Streams 跨 Pod 广播
live-go
直播推拉流
客户端层
portal-web 门户
React19 + AntD5
admin-web 管理端
knowledge-admin 知识管理端
小程序 ×2（Taro4 双端）
knowledge-mp / student-mp
H5 ×2
student-h5 / knowledge-h5
api.edu.com
REST 统一入口
共享库（横切全部服务）
ep-web-common
租户Filter/RoleChecker
Snowflake ID/虚拟线程信号量
ep-disruptor
多级流水线
ep-cache
三语言统一缓存
ep-data
读写分离+双DB路由
BFF 聚合层
portal-bff
Netty Edge Server + RingBuffer(65536)
聚合路由 / 蓝权 / 灰度 / 幂等
CellRouter 多活单元路由
knowledge-bff
知识域聚合网关
错误翻译注入
Dubbo Triple RPC
Dubbo Triple RPC
AI 层（Python 3.12 · Litestar）
ai-platform（9 模块）
ai_gateway / ai_rag / ai_agent
ai_mcp / ai_a2a / ai_harness
ai_hermes / ai_plugin / ep_memory
业务服务层（Java 21 · 虚拟线程 · 14 服务）
core-business
课程/用户/报名
order-platform
订单中台
grading-service
作业批改
senior-care
康养监护
payment-platform
支付中台
boardgame-service
棋牌对弈
knowledge-service
retrieval-service
检索
agent-service
Agent 编排
mcp-server
tenant-service
多租户+SaaS 控制平面
data-platform
数据平台
中间件层（K8s 集群内存储）
Milvus
向量库
CockroachDB
海外单元
Redis Cluster
TiDB
主库 HTAP
Pulsar
事件总线
NebulaGraph
→ TiDB CTE 兜底
Seata
Saga 协调

分层要点：

- **接入层**：Traefik Ingress 暴露 3 个域名 —— `api.edu.com`（REST 统一入口）、`www.edu.com`（门户静态站）、`ws.edu.com`（WebSocket 长连接）
- **BFF 层**：`portal-bff` 是自研 Netty 边缘服务器（65536 RingBuffer 削峰），内置 **CellRouter** 多活单元路由（Hash 路由 + WriteHoldRegistry 写保持，D-083/D-090）；`knowledge-bff` 聚合知识域
- **服务层**：Java 业务服务全部运行于虚拟线程；跨服务同步调用走 Dubbo Triple，异步走 Pulsar 领域事件
- **中间件层**：生产中间件一律 K8s 集群内部署（FQDN 访问），禁止 localhost（铁律 1 类约束）

2.2 服务拓扑与领域划分



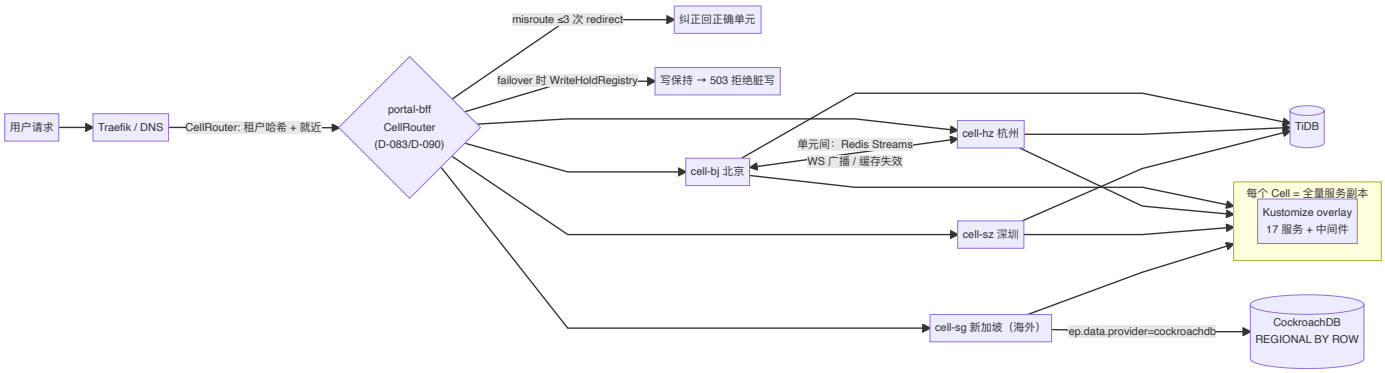


服务清单（17 个后端服务 + 7 个公共库）：

#	服务	语言	领域	职责
1	`core-business`	Java	教学	课程/用户/报名，领域事件 + Disruptor 副作用流水线
2	`grading-service`	Java	教学	作业批改，Judge0 沙箱 + 三级流水线
3	`order-platform`	Java	交易	订单中台（api/service 多模块），Seata Saga
4	`payment-platform`	Java	交易	支付中台（api/service 多模块），回调幂等流水线
5	`portal-bff`	Java	平台	统一 BFF：Netty Edge + CellRouter + 灰度
6	`knowledge-bff`	Java	知识	知识域聚合网关
7	`knowledge-service`	Java	知识	文档/语料管理（MyBatis-Plus）
8	`retrieval-service`	Java	知识	检索服务（Milvus）
9	`agent-service`	Java	知识	Agent 编排（Java 侧）
10	`tenant-service`	Java	SaaS	多租户 + Metering/Billing/ApiKey 控制平面
11	`mcp-server`	Java	知识	MCP 工具服务器
12	`data-platform`	Java	数据	血缘/剖析/漂移检测/NL Query/AIOps
13	`senior-care`	Java	康养	IoT 遥测异常检测告警
14	`boardgame-service`	Java	棋类	象棋规则引擎 + Elo 天梯 + 对局回放
15	`ws-gateway`	Go	实时	纯 stdlib RFC6455 WebSocket 网关
16	`live-go`	Go	实时	直播推拉流 / 弹幕批量管道
17	`ai-platform`	Python	AI	9 模块：gateway/rag/agent/mcp/a2a/harness/hermes/plugin/memory

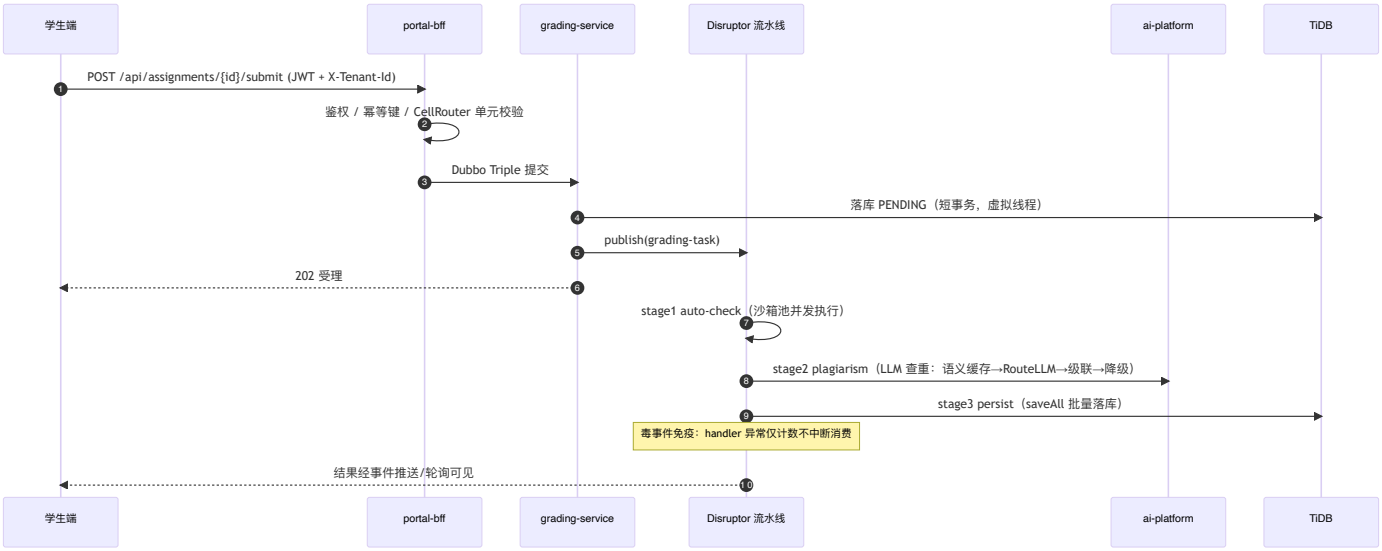
公共库	语言	提供能力
`ep-web-common`	Java	TenantContextFilter 租户过滤器、RoleChecker、**Snowflake ID（时钟回拨保护）**、DbConnectionSemaphore 虚拟线程信号量、全局异常/Actuator 安全
`ep-disruptor`	Java	LMAX Disruptor 单级 Worker + 任意级 Pipeline + Registry 生命周期（§3.1）
`ep-cache`	Java	L1 Caffeine + L2 Redis + Pulsar 失效广播（三语言协议对齐）
`ep-data`	Java	读写分离路由 + TiDB/CockroachDB 双 provider（§3.2）
`ep-memory-java`	Java	四级记忆体系 Java 侧（L0~L3）
`ep-common-go`	Go	缓存失效协议等 Go 侧公共件
`ep-common-python`	Python	缓存失效协议等 Python 侧公共件

2.3 多活单元（Cell）架构



- **4 个多活单元**（bj / hz / sz / sg），Karmada propagation 跨集群调度，每单元独立 Kustomize overlay + TopologySpread
- **CellRouter**：Hash 路由 + misroute 纠正（≤3 次 redirect，>3 次 reject）+ **WriteHoldRegistry**（failover 期间写保持，防跨单元脏写）；路由状态外化 Redis（铁律 1）
- **cell-sg 海外单元**走 CockroachDB（`REGIONAL BY ROW` 多区域表），国内走 TiDB —— 由 `ep-data` 按 `ep.data.provider` 属性一键切换（§3.2，D-194/D-203）

2.4 请求链路剖面：一次「提交作业 → AI 批改」



这条链路同时体现了四个核心设计：BFF 边缘削峰（RingBuffer 65536）、批改流水线（Disruptor 三级）、AI 成本栈（四层）、以及「重 IO 移出 DB 事务」原则。

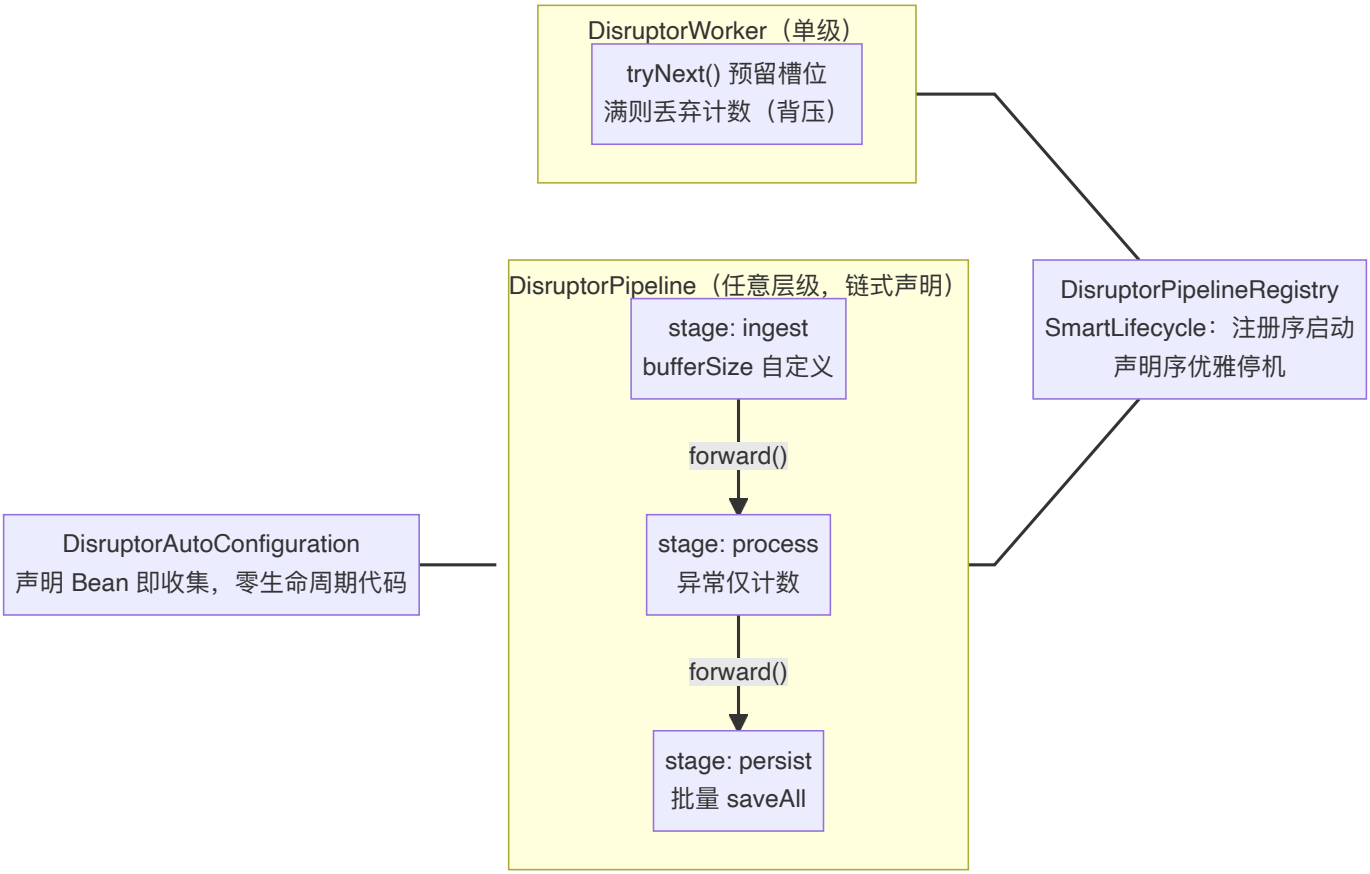
---

3. 核心设计决策

> 所有重大决策均按**铁律 8**完成 ≥3 方案对比，登记于 `decisions.jsonl`（D-001 ~ D-203），关键决策另有 ADR 文档（`docs/adr/`）。

3.1 LMAX Disruptor 多级流水线（全服务共用的高并发内核）

这是本项目最有辨识度的并发设计。 公共库 `services/ep-disruptor` 提供两类原语：



工程化加固（全部有测试锁定）：

特性	实现	为什么重要
自定义层级数	`pipeline.stage(...)` 链式声明 N 级，`forward()` 自动路由	各服务按负载形态定制 1~N 级
毒事件免疫	内置 ExceptionHandler：handler 抛异常 → log + `failedCount()` 计数 → 消费线程不退出	默认 FatalExceptionHandler 会永久毒化 RingBuffer
槽位不毒化	translator 抛异常仍 publish 保留序列	杜绝「预留不发布」死槽
优雅停机	pipeline 内**声明序** shutdown（上游先 drain）	停机不丢在途事件
即插即用	服务声明 `DisruptorPipeline` Bean 即被自动收集	零生命周期模板代码

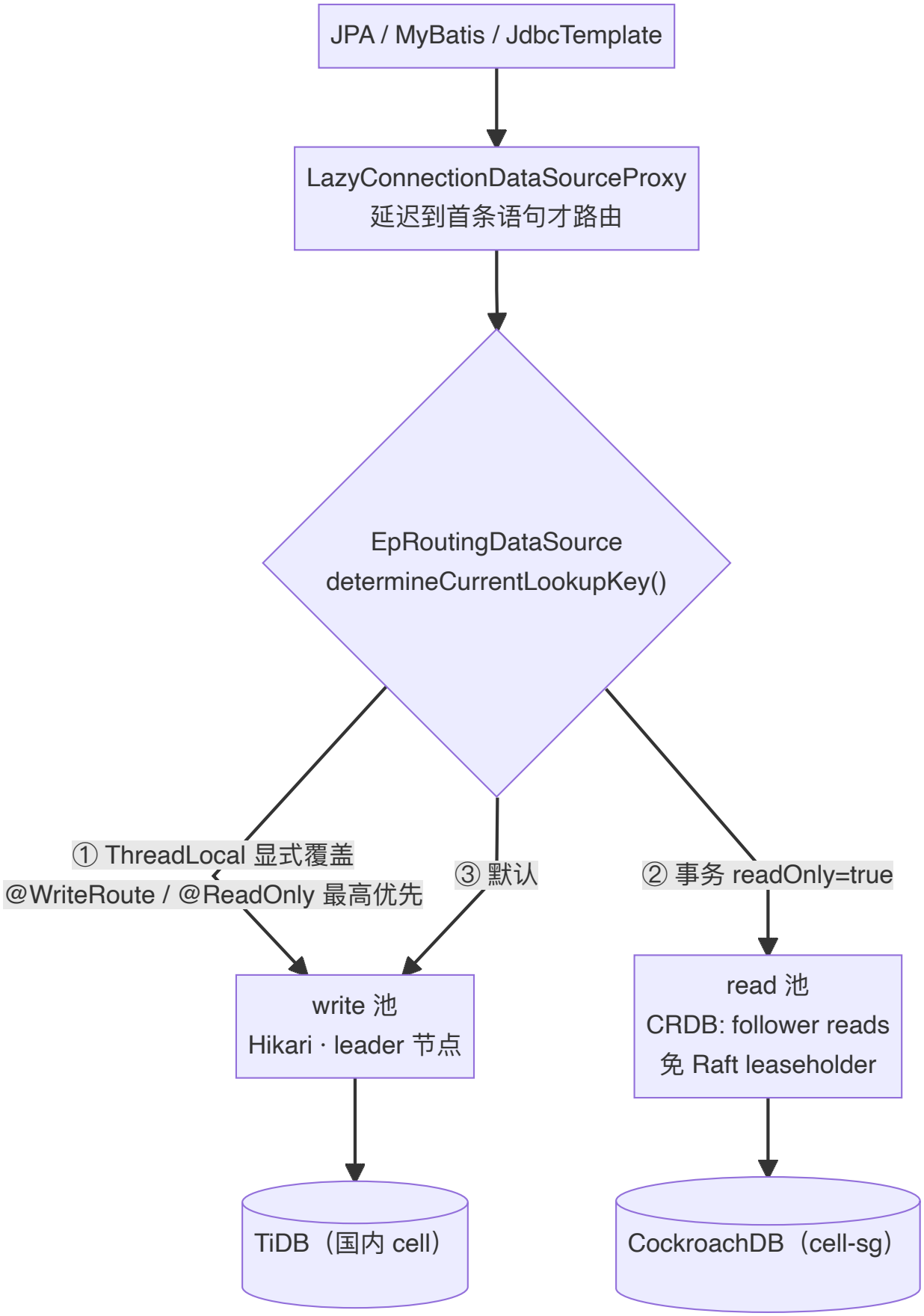
典型服务流水线分配：

服务	流水线	解决的问题
`grading-service`	`grading-task → auto-check → persist`	把 10s 级沙箱执行 + Dubbo 查重挪出 DB 事务
`payment-platform`	`callback → verify(SETNX 幂等) → settle`	渠道回调高频削峰
`order-platform`	`order-write → event-publish`	事务内逐条领域事件 → 批量发布（outbox 化）
`boardgame-service`	`move → rating-update → replay-persist`（BUSY_SPIN）	走子高频写路径脱离逐条落库
`senior-care`	`sample → anomaly-detect → alert-persist`	IoT 遥测窗口聚合批量告警
`core-business`	`course-cmd → side-effects`	课程发布事件 + 缓存失效异步化
`knowledge-service`	`upload → parse → vectorize`	文档解析 → 批量 embedding
`data-platform`	`cdc-ingest → normalize → batch-writer`（65536）	CDC 摄入批量落库 + Pulsar 降级
`portal-bff`	`gateway-ingress`（65536）	Netty 边缘写路径削峰

3.2 双数据库架构：TiDB 主库 + CockroachDB 海外（ep-data）

公共库 `services/ep-data` 让**每个服务自由选择存储 provider 并自动获得读写分离**，一个属性切换：

```yaml
ep.data:
  enabled: true
  provider: cockroachdb   # tidb | cockroachdb | mysql | h2 —— 服务级自由选择
  read-replicas: true
  follower-reads: true    # CRDB 专属：读池自动 SET default_transaction_use_followers
  write: { host: crdb:26257, database: ep_business, maximum-pool-size: 10 }
  read:  { host: crdb-ro:26257, database: ep_business, maximum-pool-size: 20 }
```



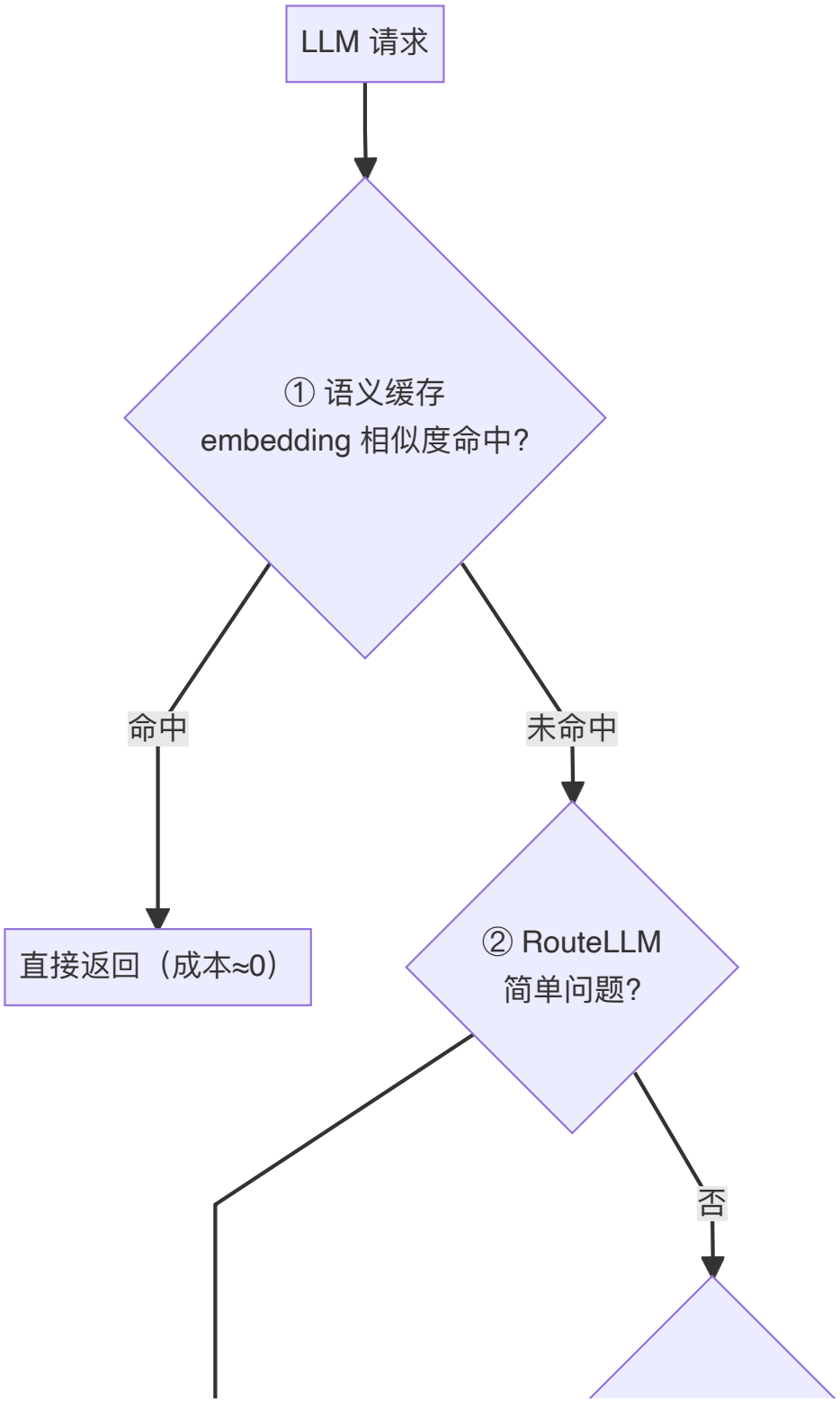
设计要点：

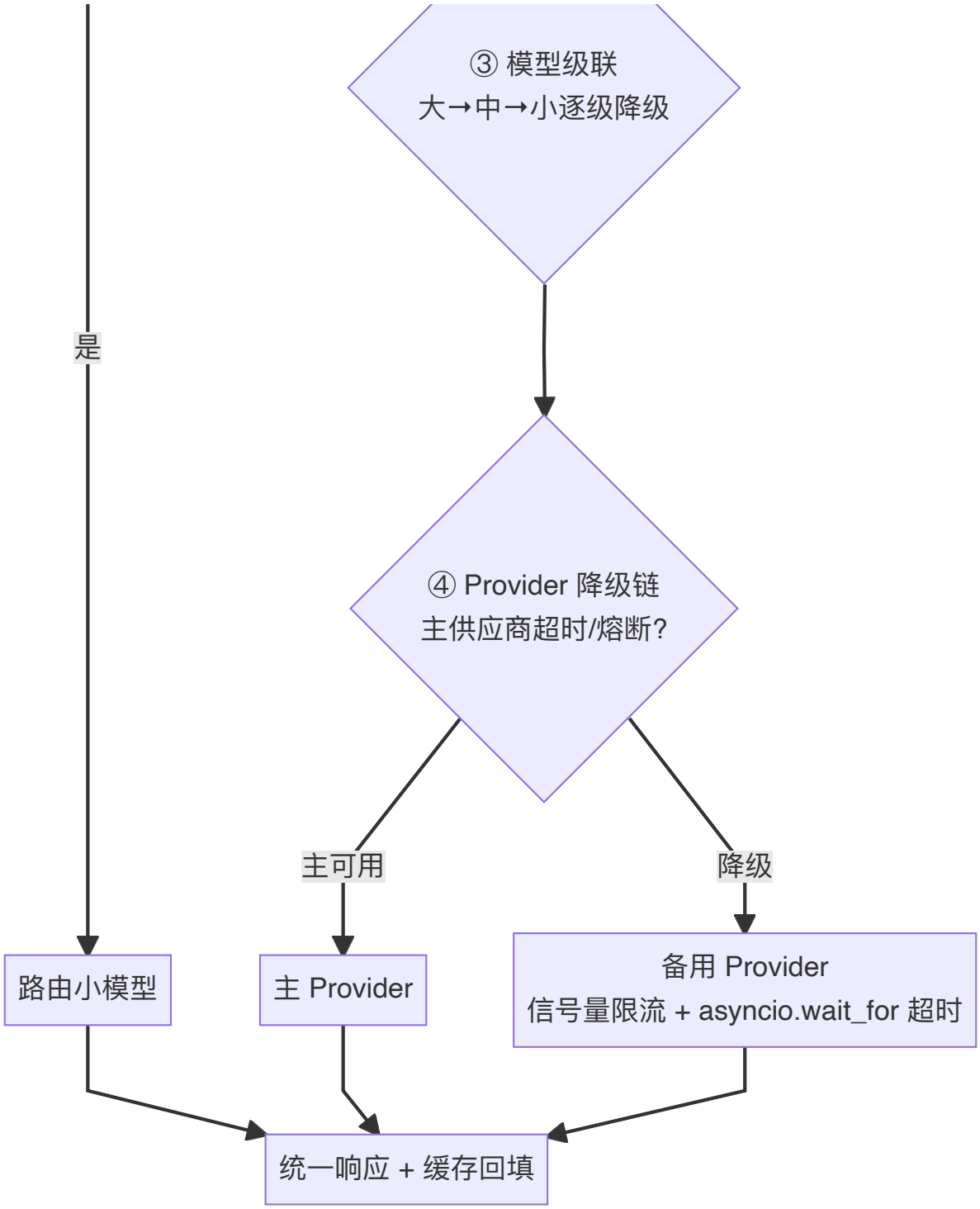
- **三级路由决策链**：显式覆盖 > 事务只读标志 > 写库 —— `@WriteRoute` 解决「写后读一致性」（lagging replica 上读不到自己刚写的）

- CRDB 读池连接级 `SET default_transaction_use_followers = on`（stale follower reads，只读事务专用）+ `ApplicationName` 打通 CRDB 控制台可观测
- H2 / 无副本 provider 自动把 READ 折叠到写池（同一 Bean 图，零调用方特判）—— 本地单机测试与生产双库共用一套代码路径
- **双库选型经 PoC 评审**：D-194（CockroachDB PoC 结果）+ D-203（CRDB vs TiDB 替换评估），另有操作手册 `docs/architecture/crdb-dual-db-guide.md`

3.3 AI 平台四层成本栈（Python）

`ai-platform` 的 LLM 调用统一经过四层成本治理，自上而下逐层兜底：





配套治理：

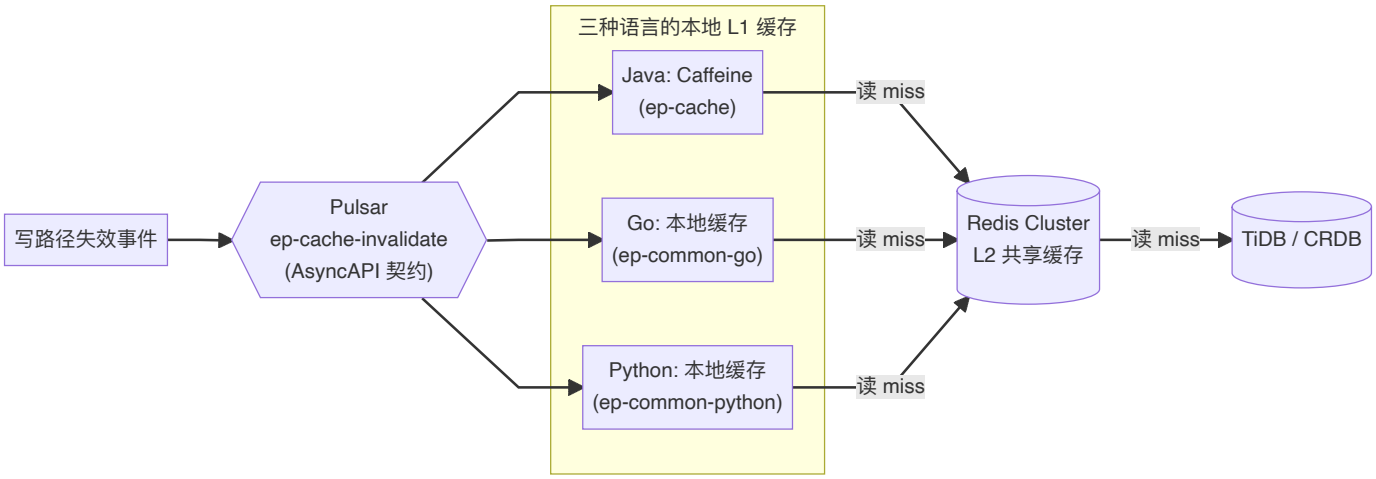
- **LLM 并发治理**：全局 + 每 Provider `asyncio.Semaphore`；同步 Redis/LLM 调用全部 `asyncio.to_thread` 化，事件循环零阻塞
- **四级记忆体系**（`ep-memory`，Java/Python 双语言实现）：L0 即时 → L1 工作记忆 → L2 情景 → L3 语义，Mem0 集成
- **Mock 模式铁律**：`LLM_MODE=mock`（默认）返回确定性响应，CI 不依赖外部 API
- **Agent 架构**：MCP（工具调用）+ A2A（Agent 间协作）+ LangGraph 状态机 + Reflexion 自我反思循环

3.4 多租户与 SaaS 控制平面

能力	实现	位置
租户上下文	`TenantContextFilter` 解析 `X-Tenant-Id` → ThreadLocal 透传	`ep-web-common`
SQL 隔离	MyBatis TenantLine 拦截器自动追加租户条件 + Row-Level 隔离测试	各服务
计量 Metering	`UsageRecord` 用量采集 → 计量服务	`tenant-service`
计费 Billing	`BillingInvoice` 账单生成	`tenant-service`
API Key	`ApiKey` 实体 + 校验（租户级开放能力）	`tenant-service`
单元路由	租户哈希 → Cell 映射（多活）	portal-bff CellRouter

设计动机与迁移路径见 `docs/architecture/saas-multi-tenant-transformation.md`（D-198 租户上下文下沉公共库）。

3.5 三语言缓存一致性协议（ep-cache）



- 失效广播协议在 `docs/contracts/asyncapi/ep-cache-invalidate.yaml` 有正式契约，三种语言实现互验（契约测试）
- L1（进程内）+ L2（Redis）两级；写路径先落库再广播失效，读路径 L1 → L2 → DB

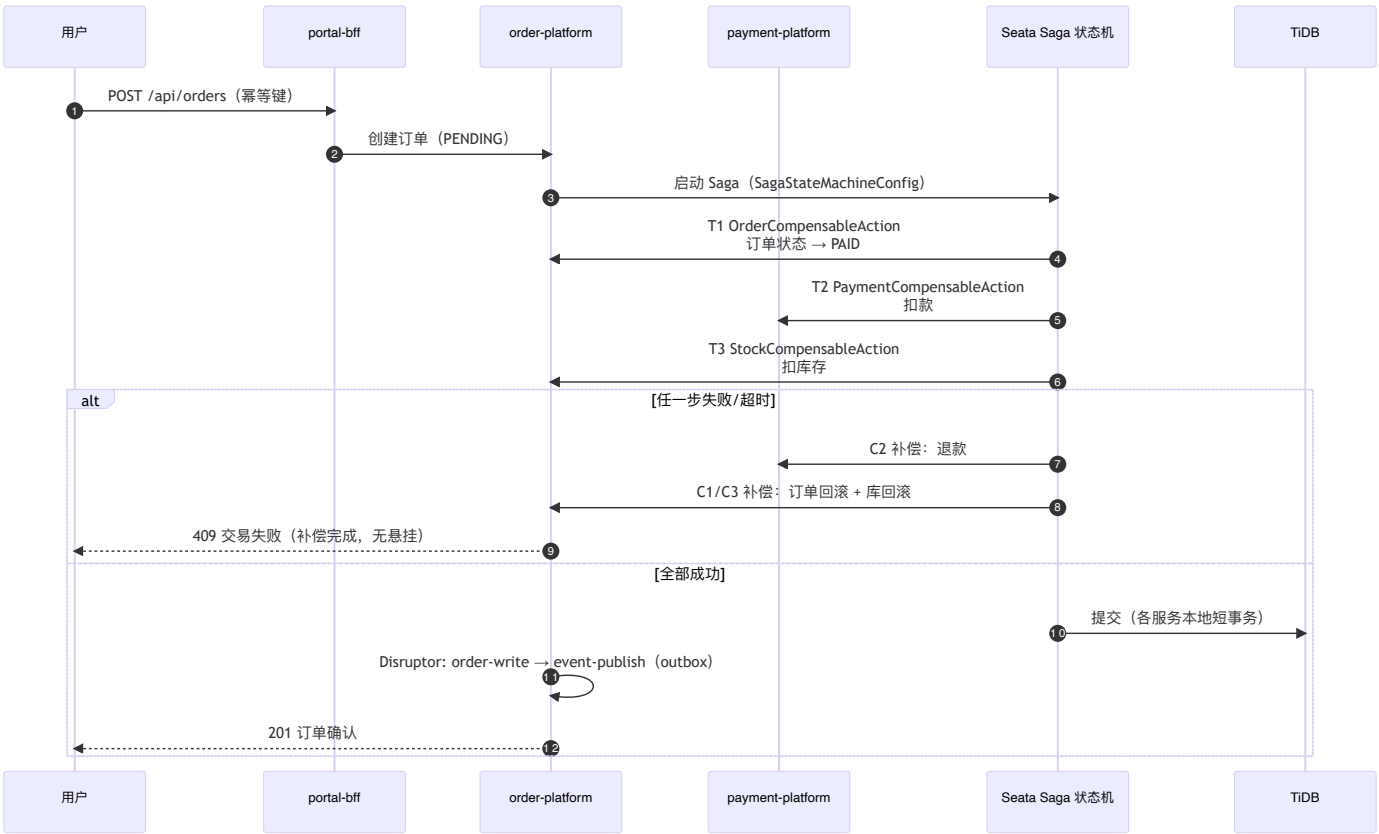
3.6 其他技术亮点速览

领域	设计	关键位置
BFF 边缘	Netty Edge Server + 65536 RingBuffer 入队 + 批量转发器（异步 sendAsync + 虚拟线程池 + 信号量）	`services/portal-bff`
WebSocket	Go 纯标准库 RFC6455（帧重组 + 心跳 deadline）；跨 Pod 广播经 Redis Streams 消费组（at-least-once）；生产 fail-closed（无 JWT 公钥拒启）	`services/ws-gateway`
ID 生成	Snowflake ID + **时钟回拨保护**（AtomicSnowflakeIdGenerator）	`ep-web-common`
棋类引擎	象棋规则引擎（走子合法性/将军判定）+ Elo 天梯 + Stockfish sidecar	`services/boardgame-service`
图谱	NebulaGraph 主路径 + TiDB CTE 递归查询兜底（图库故障不中断血缘/图谱能力）	`data-platform` / `retrieval-service`
安全	JWT RS256 双密钥、Actuator Basic 认证、Semgrep + Trivy + Aikido 三重扫描	`.github/workflows/`
AOT	面向 GraalVM Native Image：禁反射/运行时资源加载，native-ready 脚本门禁	`scripts/check-native-ready.sh`

4. 关键实现剖析

4.1 订单支付 Saga（数据流）

下单涉及订单/支付/库存三个服务的长事务，采用 **Seata Saga 状态机**编排，每一步正向操作配补偿操作：



实现要点（`services/order-platform/order-platform-service/.../saga/`）：

- **三个补偿动作**：`OrderCompensableAction` / `PaymentCompensableAction` / `StockCompensableAction`，状态机由 `SagaStateMachineConfig` 声明
- **补偿链有集成测试锁定**：`SagaE2eIntegrationTest`、`OrderPaymentSagaIT`、`SagaCompensationIntegrationTest`、`SeataGlobalRollbackIntegrationTest`（含全局回滚验证）
- **幂等**：BFF 幂等键 + 支付回调 SETNX；`IdempotencyAndOptimisticLockTest` 锁定
- **事件外发**：Saga 完成后领域事件经 Disruptor `event-publish` 流水线批量发 Pulsar（事务外 outbox 化，`order-service-events.yaml` 契约）

4.2 知识库 RAG 管道

```
文档上传 → knowledge-service（upload → parse → vectorize 三级流水线）
        → retrieval-service（Milvus 向量检索 + FGAC 租户细粒度过滤）
        → agent-service（编排）→ ai-platform（ai_rag 生成，四层成本栈）
```

- 向量化走 `knowledge-service` 的 Disruptor 批量 embedding（避免逐文档同步调用）
- 检索侧租户隔离（FGAC）：向量过滤条件强制携带 tenant_id
- 生成侧命中 §3.3 成本栈，命中语义缓存的重复提问成本趋近于零

4.3 实时互动（Go）

组件	实现要点
`ws-gateway`	纯标准库 RFC6455：握手/帧重组/心跳 deadline 全手写；JWT 鉴权（生产 fail-closed，`failclosed.go` 可单测）；跨 Pod 广播走 Redis Streams 消费组
`live-go`	直播推拉流信令 + 弹幕批量管道（channel 泵 + 批量 flush）
协同编辑	Yjs CRDT over WebSocket（知识域协同）

---

5. 目录结构

```
education-platform/
├── services/                          # 后端（17 服务 + 公共库）
|   ├── core-business/                 # 核心教学域（课程/用户/报名）
|   ├── grading-service/               # 作业批改（+ grading-api/ Dubbo API 模块）
|   ├── order-platform/                # 订单中台（api + service 多模块）
|   ├── payment-platform/              # 支付中台（api + service 多模块）
|   ├── knowledge-platform/            # 知识域 7 模块
|   |   ├── knowledge-service/  knowledge-bff/  retrieval-service/
|   |   ├── agent-service/  tenant-service/  mcp-server/  knowledge-api/
|   ├── senior-care/                   # 康养监护（IoT 遥测）
|   ├── boardgame-service/             # 棋牌对弈（象棋规则引擎 + Elo）
|   ├── data-platform/                 # 数据平台（血缘/剖析/漂移/NL Query/AIOps）
|   ├── portal-bff/                    # 统一 BFF（Netty + Disruptor + CellRouter）
|   ├── ws-gateway/                    # Go：纯 stdlib WebSocket 网关
|   ├── live-go/                       # Go：直播/弹幕
```

```
|   ├── ai-platform/                 # Python: AI 平台 9 模块
|   |   ├── ai_gateway/  ai_rag/  ai_agent/  ai_mcp/  ai_a2a/
|   |   ├── ai_harness/  ai_hermes/  ai_plugin/  ep_memory/  tests/
|   ├── integration-tests/           # 跨服务集成测试工程（Saga/Pact/缓存一致性/TiDB live）
|   ├── ep-web-common/               # 公共: 租户Filter/RoleChecker/Snowflake/虚拟线程信号量
|   ├── ep-disruptor/                # 公共: Disruptor 多级流水线
|   ├── ep-cache/                    # 公共: 三语言统一缓存 + Pulsar 失效广播
|   ├── ep-data/                     # 公共: 读写分离 + 双 DB 路由
|   ├── ep-memory-java/              # 公共: 四级记忆（Java 侧）
|   ├── ep-common-go/  ep-common-python/
|   └── archive/                     # 归档（ep-rw-java 旧读写分离，语义并入 ep-data）
├── frontend/                        # pnpm monorepo（Turborepo, @ep/ scope）
|   ├── apps/
|   |   ├── portal-web/              # 门户（Vite + React19 + AntD5）
|   |   ├── admin-web/               # 平台管理端
|   |   ├── knowledge-admin/         # 知识管理端
|   |   ├── student-h5/  knowledge-h5/          # H5 ×2
|   |   └── student-mp/  knowledge-mp/          # 小程序 ×2（Taro）
|   └── packages/                    # ui-kit · api-client · query · shared · lint-config
├── deploy/
|   ├── k8s/base/                    # 34 份 manifest: 17 应用 + 中间件 +
HPA/PDB/NetworkPolicy
|   ├── k8s/overlays/                # cell-bj/hz/sg/sz + local-real + local-verify
|   ├── k8s/local-real-direct/       # 本地 K3s 真实中间件直连部署
|   ├── gitops/argocd/               # app-of-apps + ApplicationSet（apps/base/cells）
|   ├── load-tests/k6/               # 12 压测脚本 + results/ 报告
|   ├── build/                       # JIB + 批量构建脚本
|   ├── sql/  test/  docs/           # DDL / compose / 扩容 runbook
├── docs/                            # 全量工程文档（见 §10 文档索引）
├── scripts/                         # 铁律扫描 / 契约 lint / E2E / 压测 RUNBOOK
└── .github/workflows/               # 14 条 CI/CD workflow
```

6. 快速开始

6.1 环境要求

依赖	版本	用途
JDK	21	全部 Java 服务
Maven	3.9+	Java 构建
Go	1.25	ws-gateway / live-go
Python + uv	3.12	ai-platform
Node + pnpm	≥22 / 9+	前端 monorepo
Rancher Desktop（K3s）	—	本地集群联调

6.2 后端启动

```
# 1. 安装公共库（其余服务编译依赖）
mvn -pl services/ep-web-common,services/ep-disruptor,services/ep-data,services/ep-cache -
am install -DskipTests

# 2. 任一服务独立运行（H2 + 内嵌依赖，无需中间件）
mvn -pl services/core-business spring-boot:run

# 3. 或 docker compose 起测试依赖
docker compose -f deploy/test/docker-compose-test.yml up -d
```

6.3 AI 平台（Python）

```
cd services/ai-platform
uv sync
LLM_MODE=mock uv run granian --interface asgi ai_gateway.app:create_app
uv run pytest -q                    # 189 个测试
```

6.4 前端

```
cd frontend
pnpm install
pnpm dev --filter portal-web        # 门户；--filter admin-web / knowledge-admin 同理
```

6.5 本地 K3s 全栈（真实中间件）

```
# TiDB / Redis / Pulsar / Milvus 部署于 K3s 集群内，服务直连
kubectl apply -f deploy/k8s/local-real-direct/all-services.yaml
```

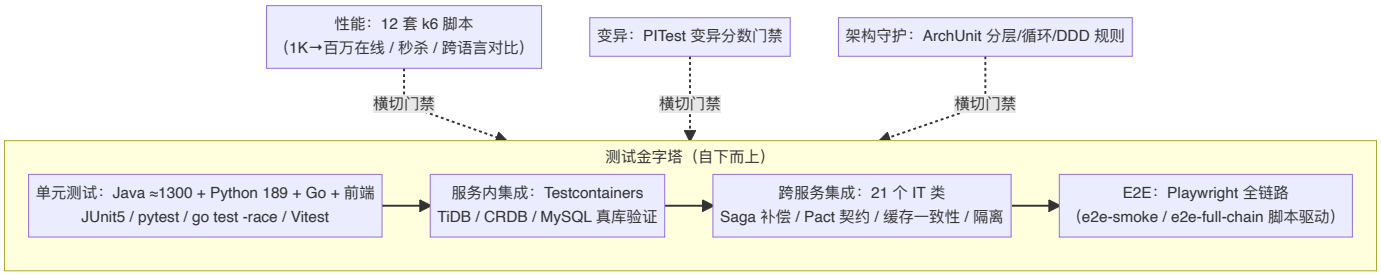
6.6 运行测试

```
mvn test                                    # Java（或逐服务）
cd services/ws-gateway && go test ./... -race
cd services/live-go && go test ./...
cd services/ai-platform && uv run pytest -q
cd frontend && pnpm test
bash scripts/run-all-tests.sh               # 全语言聚合入口
```

---

7. 测试体系

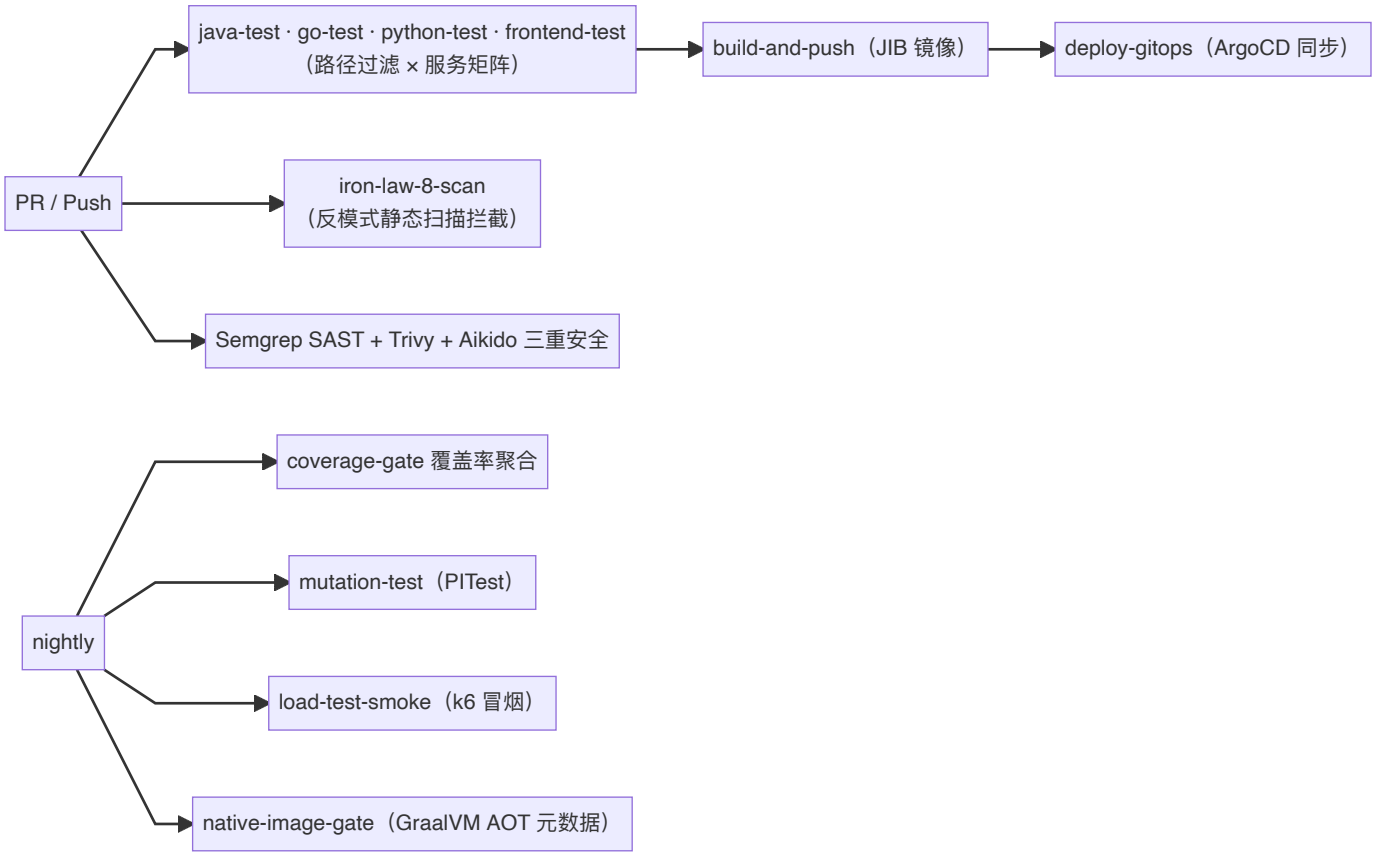
7.1 测试金字塔与实测规模



层级	规模	工具	门禁
Java 单元	**≈ 1300 @Test**	JUnit 5 + Mockito	PR 全跑
服务内集成	**21 个 IT 类**（Testcontainers：TiDB/CRDB/MySQL）	Testcontainers	PR + live 标签
跨服务集成	Saga E2E / Pact 契约 / 缓存一致性 / 租户隔离 / 幂等乐观锁	Spring dev-local + fakes	PR
Go 单元	全覆盖（含 **-race** 跑法）	testing + miniredis	PR
Python 单元	**189 测试函数**	pytest（live/testcontainers 标记）	PR
前端单元	Vitest + Testing Library	Vitest	PR
E2E	Playwright 全链路（冒烟 + 完整链路）	`scripts/e2e-*.sh`	PR + 发版前
压测	**12 套 k6 脚本** + threshold 门禁	k6	nightly + 发版前
变异测试	PITest（核心服务）	mutation score 门禁	nightly
架构守护	ArchUnit（分层/循环依赖/贫血模型）	—	PR

代表性跨服务集成测试（`services/integration-tests/`）：`SagaE2eIntegrationTest`、`OrderPaymentSagaIT`、`SeataGlobalRollbackIntegrationTest`、`CrossServiceContractTest`、`OrderPaymentPactTest` / `GradingAssignmentPactTest`（Pact 双向契约）、`CacheConsistencyTest`、`TenantIsolationE2ETest`、`TiDBIntegrationIT` / `RedisIntegrationIT`（live 标签）。

7.2 CI 质量门禁（14 条 Workflow）



7.3 压测结论（诚实标注）

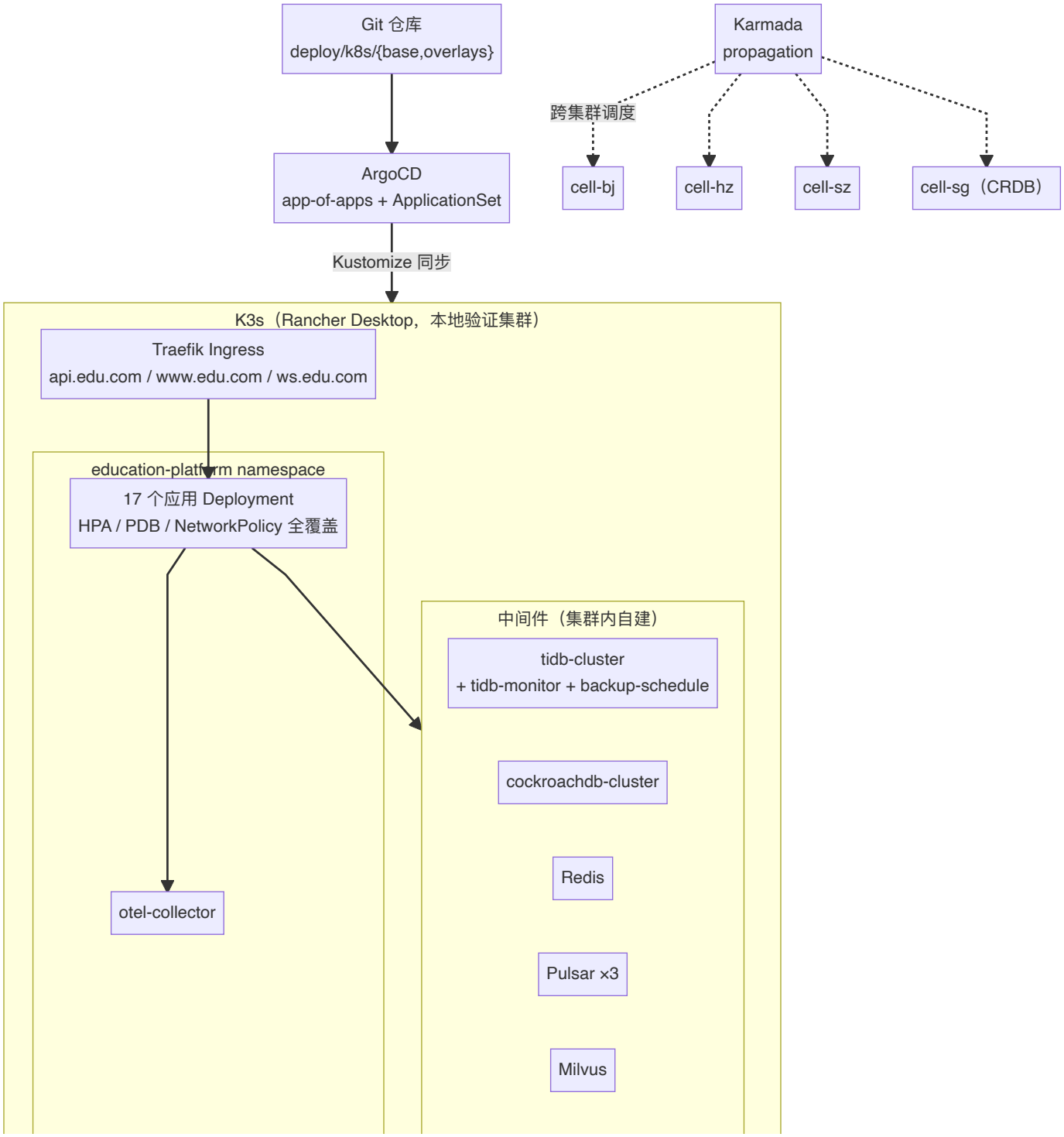
12 套脚本覆盖 1K → 10K → 100K → 百万级在线模拟、秒杀尖峰、跨语言网关对比、缓存一致性、WebSocket 连接风暴。结论要点：

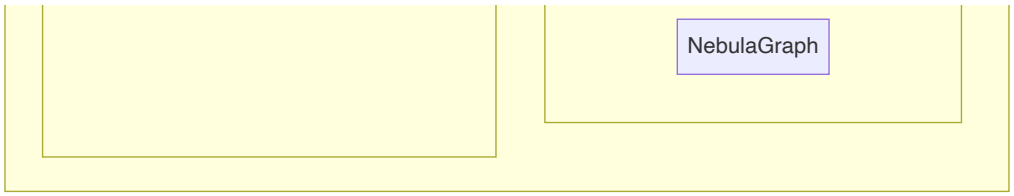
- **Go 网关韧性最佳**（cross-lang-benchmark 对比）
- 共享集群下 TiDB/TiKV 写入存在瓶颈，已按瓶颈归因落地治理：JDBC 批量参数、Disruptor 批量 flush、`DbConnectionSemaphore` 连接限流
- 完整数据见 `deploy/load-tests/k6/`（脚本）与 results 报告

---

8. 部署

8.1 部署架构（K3s 本地 / 多集群生产，4 Cell 多活）





8.2 部署形态要点

维度	方案
本地集群	K3s（Rancher Desktop），中间件全部集群内部署（铁律：禁 localhost）
镜像	**JIB 免 Dockerfile** 构建，Java 服务统一根 BOM 版本
声明式	Kustomize base（**34 份 manifest**：17 应用 + TiDB/CRDB/Milvus/Nebula CRD + HPA/PDB/NetworkPolicy/stockfish-sidecar）
多单元	overlays：`cell-bj` / `cell-hz` / `cell-sz` / `cell-sg` + `local-real` / `local-verify`
GitOps	**ArgoCD app-of-apps + ApplicationSet**（apps / base / cells 三组），声明式同步
多集群	**Karmada propagation** 跨集群调度（`karmada-propagation.yaml`）
域名	Traefik 三域名：`api.edu.com`（REST）、`www.edu.com`（门户）、`ws.edu.com`（WebSocket）

8.3 部署操作

```
# 本地 K3s 全栈（真实中间件）
kubectl apply -f deploy/k8s/local-real-direct/all-services.yaml

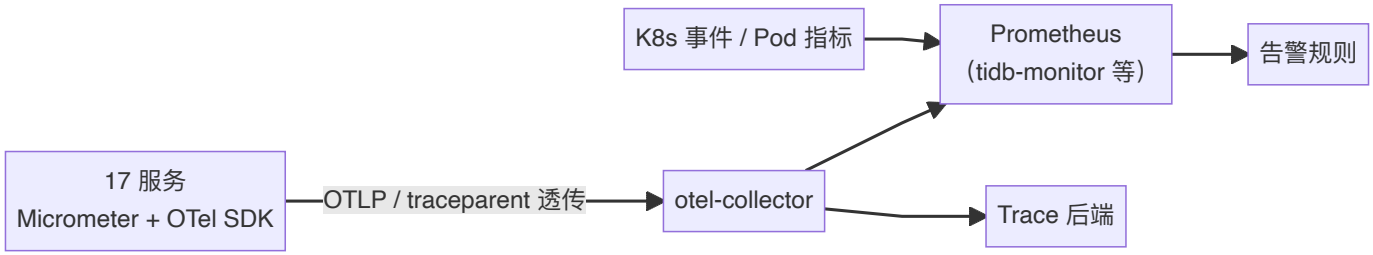
# 指定 cell overlay
kubectl apply -k deploy/k8s/overlays/local-real

# GitOps 方式（ArgoCD 接管）
kubectl apply -f deploy/gitops/argocd/app-of-apps.yaml

# 镜像构建（JIB）
mvn -pl services/core-business package -Pjib
```

9. 监控运维

9.1 可观测体系



- **全链路追踪**：`traceparent` 头跨 BFF → Dubbo → Pulsar → Go/Python 透传（`TracePropagationTest` 锁定）
- **中间件监控**：tidb-monitor、CRDB 控制台（ApplicationName 标识读写池）
- **Disruptor 指标**：各流水线 failedCount / 槽位丢弃计数暴露为 Micrometer 指标
- **生产安全基线**：Actuator Basic 认证、JWT 生产 fail-closed（无公钥拒启）

9.2 运维手册

场景	文档
生产运行手册	`docs/deployment/production-runbook.md`
回滚预案	`docs/deployment/rollback-plan.md`
上线检查清单	`docs/deployment/go-live-checklist.md`
多活验证	`docs/deployment/multi-active-verification.md`
局域网联调	`docs/deployment/lan-test-runbook.md`
TiDB 扩容	`deploy/docs/tidb-scaling-runbook.md`
各阶段部署细节	`docs/deployment/sp01~sp07-deploy.md`

常用运维脚本：`scripts/port-forward-infra.sh`（中间件端口转发）、`scripts/live-integration-check.sh`（live 环境健康检查）、`scripts/e2e-smoke-test.sh`（冒烟）。

---

10. 文档索引

类别	数量	入口
需求 SRS	7	`docs/requirements/sp01-srs.md` ~ `sp07-srs.md`
架构设计 SAD	7+	`docs/architecture/sp01-sad.md` ~ `sp07-sad.md`
详细设计 SDD	8	`docs/design/sp01-sdd.md` ~ `sp07-sdd.md`（含 `sp06-contract-testing-design.md`）
API 契约（OpenAPI）	9	`docs/contracts/openapi/`：core-business / grading / order / payment / portal-bff / knowledge-bff / knowledge-service / tenant / data-platform
事件契约（AsyncAPI）	8	`docs/contracts/asyncapi/`：core-business 领域事件 / grading / order / knowledge / tenant / senior-care / boardgame / **ep-cache-invalidate**
ADR 决策	14 篇 + 203 项登记	`docs/adr/`（D-113~D-203 等）+ `.zcode/sessions/education-platform/decisions.jsonl`
架构评分卡	v5	`docs/architecture/architecture-scorecard.md`（94.5/A）
双数据库指南	—	`docs/architecture/crdb-dual-db-guide.md`
SaaS 多租户改造	—	`docs/architecture/saas-multi-tenant-transformation.md`
需求追溯矩阵	—	`docs/architecture/requirements-traceability-matrix.md`（铁律 10）
测试文档	19	`docs/testing/`（comprehensive-test-report / cross-service-integration-report / e2e-full-chain-report / production-readiness-report 等）
部署文档	13	`docs/deployment/`
TOGAF 映射	9	`docs/togaf/`
编码规范	—	`docs/coding-standards.md`
中台复用评估	—	`docs/middle-platform-reusability-assessment.md`
压测	12 脚本	`deploy/load-tests/k6/`
项目铁律	10 条	`CLAUDE.md` §6

契约治理：`.spectral.yml` lint 门禁 + `scripts/lint-openapi.sh` / `lint-asyncapi.sh` / `validate-api-contracts.py`；契约为 source-of-truth，运行时 drift 检测。

---

11. 项目治理：铁律与决策

11.1 十条工程铁律（CLAUDE.md 摘要）

#	铁律	要点	决策号
1	会话状态外化	全部写 Redis Cluster，禁止进程内 Map 充当持久化	D-006
2	读写分离	读走只读副本、写走主库，强制 `@ReadOnly` / `@WriteOnly`	—
3	缓存铁律	先 Redis 后 DB，禁止裸查库	D-033
4	消息铁律	跨服务消息走 Pulsar + Avro schema，禁止裸 JSON	—
5	先调研后落地	新功能必须先检索官方最佳实践，禁止凭记忆编码	—
6	Dubbo + SCA	RPC=Dubbo 3.3，禁 Netflix 组件	—
7	虚拟线程 + GraalVM AOT	Java 全量虚拟线程；禁反射/运行时资源加载	D-039/D-040
8	**决策对比分析**	技术决策实施前 ≥3 方案对比（含自研时必须列成熟开源），记入 decisions.jsonl	D-048
9	**运行验证真相律**	制品存在 ≠ 能跑；所有「完成」以运行证据为准	D-140
10	**设计-实现可追溯**	SRS→ADR→代码→测试全链路追溯，CI 脚本扫描	D-196

铁律 8/9/10 均有 CI 执行体：`scripts/check-iron-law-8.py`、`check-iron-law-9.py`、`check-iron-law-10.py`。

11.2 决策留痕

- **203 项决策**（D-001 ~ D-203）登记于 `.zcode/sessions/education-platform/decisions.jsonl`，含问题/备选方案/对比维度/结论
- 代表性决策：D-113 Seata Saga 迁移、D-114 ArgoCD GitOps、D-194 CockroachDB PoC、D-197 缓存失效统一 Pulsar、D-198 租户上下文下沉、D-199 根 BOM + ArchUnit、D-203 CRDB vs TiDB 替换评估
- 架构评分卡（`docs/architecture/architecture-scorecard.md`）：**94.5 / 100（A）**

架构评分 94.5/A · 203 项决策留痕 · 10 条铁律 · 17 份契约先行 · 1300+ Java 测试

本仓库为工程实践参考实现，持续重构演进中。